

Semana 5 — Contrato Interactable e Signals

Semana 5

Contrato Interactable e Signals

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade II — Construir Sistemas (Semanas 4–7)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender o contrato `Interactable` como mecanismo de comunicação desacoplada entre Player e objetos do mundo
- Compreender Signals como padrão observer para reação a eventos de interação
- Implementar um objeto interativo concreto (porta ou equivalente) usando contrato Interactable + Signal

Semana 5 — Contrato Interactable e Signals

ENCONTRO 1

O Contrato Interactable

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 4 (`GameManager` e `SaveManager` como Autoload) (15 min)
- Introdução: o problema da comunicação acoplada entre sistemas (20 min)
- Demonstração: criação do contrato `Interactable` e implementação em uma Scene (`Door.tscn`) (35 min)
- Laboratório: cada estudante cria seu próprio contrato e implementa uma Scene interativa (45 min)
- Desafio: adaptar a reação da Scene interativa (15 min)
- Feedback e fechamento (5 min)



**Se o Player precisar reconhecer uma porta,
uma alavanca e um baú, o que muda no
código do Player a cada novo tipo?**

Pense em como esse código cresceria a cada objeto interativo novo.

O Problema da Comunicação Acoplada

- Toda engine com múltiplos sistemas independentes precisa resolver o mesmo problema: como um sistema chama outro sem depender do tipo concreto dele
- Se o Player conhecesse `Door`, `Lever`, `Chest` explicitamente, cada objeto novo exigiria alterar o Player
- Esse acoplamento não escala — e se tornaria insustentável já na Semana 10 (interação conectada ao Inventário)

O Contrato Interactable

- Define apenas um comportamento esperado: responder a `interact()`
- Não exige hierarquia de herança comum entre os objetos que o implementam
- Godot expressa isso via **duck typing** (`has_method("interact")`) ou nó de interface do Orchestrator
- Qualquer Node pode implementar o contrato sem herdar de uma classe base

Contrato de Comportamento no Godot × Unity

Godot

- Duck typing: `has_method("interact")`
- Checagem em tempo de execução
- Mais flexível, sem exigir declaração formal

Unity

- Interface formal de C#: `IInteractable`
- Checagem em tempo de compilação
- Exige que o `MonoBehaviour` declare a interface

Demonstração — Contrato e Scene Door

O que será construído:

- Contrato `Interactable` via `has_method("interact")`
- Scene `scenes/interactables/Door.tscn`, script `door.gd` com `class_name Door`
- Função `interact()` implementada, reagindo com uma confirmação simples

Por quê: primeira Scene interativa do Vertical Slice, base de todo objeto interativo até a Semana 17.

Imagem sugerida

*Objetivo: mostrar a estrutura de pastas do projeto com **scenes/interactables/** e **scripts/components/** em destaque.*

Enquadramento: captura de tela do FileSystem Dock do Godot, árvore de pastas expandida.

*Elementos presentes: pastas **scenes/interactables/**, **scripts/components/**, **scripts/autoload/** (já existente).*

Destaque visual: as duas pastas novas desta semana.

Legenda sugerida: "Novas pastas do projeto para interativos e Components, Semana 5."

Arquitetura — Detectando o Interactable

- `InteractionComponent`: novo Component do Player, dedicado a detectar objetos próximos
- Usa `Area3D` para detecção de proximidade
- Ao receber a ação `interagir`, checa `has_method("interact")` antes de chamar
- Nunca escreve `if objeto is Door` — apenas confirma que o objeto responde ao contrato

Laboratório — Contrato e Scene Interativa

Cada estudante replica, no próprio projeto:

1. Pasta `scenes/interactables/` criada, conforme `PROJECT_ARCHITECTURE.md`
2. Scene `Door.tscn`, com script `door.gd` implementando `interact()`
3. `InteractionComponent` no Player, detectando via `Area3D`
4. Ação `interagir` adicionada ao Input Map
5. Teste de `interact()` com pelo menos duas instâncias de `Door.tscn`

Boas Práticas — Interactable

- Manter a lógica de reação dentro da função `interact()`, nunca fora dela (por exemplo, no `_process()`)
- Nomear a Scene pela função que representa (`Door.tscn`), nunca pela implementação
- Manter o `InteractionComponent` limitado a detectar e chamar o contrato — nenhuma lógica específica de `Door`, `Lever` ou `Chest` deve viver ali
- Testar sempre com pelo menos duas instâncias diferentes antes de considerar a etapa concluída



Desafio — Encontro 1

Adapte a reação da própria Scene interativa a um comportamento não demonstrado em aula — por exemplo, alternar entre dois estados visuais, ou emitir uma mensagem de depuração própria.

OBJETIVOS

Mantenha o contrato `Interactable` como único ponto de entrada — a reação continua vivendo dentro de `interact()`.

Fechamento — Encontro 1

- Contrato `Interactable` definido e implementado por ao menos uma Scene do Vertical Slice
- `InteractionComponent` detectando e chamando `interact()` sem conhecer o tipo concreto do objeto
- Desafio de reação própria implementado
- Próximo passo: Signals conectando a interação a uma reação desacoplada, no Encontro 2

Semana 5 — Contrato Interactable e Signals

ENCONTRO 2

Signals e o Objeto Interativo do Grupo

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (contrato `Interactable` implementado em uma Scene) (10 min)
- Introdução: Signals como padrão observer (20 min)
- Demonstração: declaração de um Signal de interação e conexão a uma função de reação (35 min)
- Laboratório e Desafio: cada grupo implementa seu objeto interativo com contrato + Signal (50 min)
- Feedback formal sobre as soluções apresentadas (20 min)



Se três sistemas diferentes precisassem reagir à interação com a porta, a função `interact()` deveria chamar os três diretamente?

Pense em abrir a porta, tocar um som e atualizar o GameManager ao mesmo tempo.

Signals como Padrão Observer

- O contrato `Interactable` resolve "como um objeto responde a uma chamada"
- Signals resolvem um problema complementar: "como um objeto avisa que algo aconteceu, sem saber quem está ouvindo"
- Padrão **observer**, presente em praticamente toda engine de jogos moderna
- No Godot, um Signal é declarado na Scene emissora e conectado a funções de reação, sem que o emissor conheça o receptor

Padrão Observer no Godot × Unity

Godot

- Signals: mecanismo de primeira classe do editor
- Declarado, listado e conectado visualmente em qualquer Node

Unity

- `UnityEvent` (Inspector) ou `event / Action` de C# (código)
- Resultado depende de qual convenção a equipe escolheu adotar

Demonstração — Signal interacted na Door

O que será construído:

- Signal `interacted` declarado em `door.gd`
- `interact()` passa a emitir o Signal em vez de reagir diretamente
- Função de reação separada (`_abrir_fechar()`), conectada ao Signal pela subaba **Signals**

Por quê: separa "avisar que a interação ocorreu" de "reagir à interação" — a mesma separação que sustentará a conexão ao Inventário na Semana 10.

Imagem sugerida

*Objetivo: mostrar o painel Node do Godot com o Signal **interacted** declarado na Scene **Door.tscn** e sua conexão a uma função de reação.*

*Enquadramento: captura de tela da aba **Node > Signals** do editor Godot, com **Door.tscn** aberta.*

*Elementos presentes: lista de Signals do nó, o Signal **interacted** customizado, o ícone de conexão ativa.*

*Destaque visual: o Signal **interacted** e a seta indicando a função conectada.*

Legenda sugerida: "Signal interacted declarado na Door e conectado à função de reação."

Arquitetura — Interactable + Signal

- `interact()` (contrato): recebe a chamada do `InteractionComponent`
- `interacted` (Signal): avisa que a interação ocorreu, sem saber quem reage
- Função de reação separada: conectada ao Signal, nunca chamada dentro de `interact()`
- Qualquer novo objeto interativo repete essa mesma estrutura, sem alterar o `InteractionComponent`

Boas Práticas — Signals

- Manter a lógica de reação sempre em uma função separada da função do contrato, mesmo na mesma Scene
- Nomear Signals no padrão `substantivo_verbo_passado` (`interacted` , `item_collected`)
- Conectar sempre pela subaba **Signals** — um Signal declarado mas não conectado não produz efeito visível
- Mostrar a conexão via código como alternativa, útil para objetos criados dinamicamente

Laboratório e Desafio — Objeto Interativo do Grupo

Cada grupo implementa um objeto interativo (porta ou equivalente) usando contrato `Interactable` + `Signal`:

- Scene própria em `scenes/interactables/`, com collider, `MeshInstance3D` e script com `class_name`
- Signal próprio (ex.: `lever_pulled`), emitido dentro de `interact()`
- Função de reação separada, conectada ao Signal
- Liberdade de mecanismo de acionamento: alavanca, chave, proximidade



Desafio — Encontro 2

Cada grupo implementa um objeto interativo (porta ou equivalente escolhido pelo grupo) usando contrato `Interactable` + Signal, com liberdade de mecanismo de acionamento (alavanca, chave, proximidade).

OBJETIVOS

Entrega: Feedback formal sobre as soluções apresentadas, avaliado pela Rubrica 2 — Desafios Técnicos.

Checkpoint — Base do Módulo 2

Ao final da semana, cada estudante/grupo possui:

- Contrato `Interactable`, implementado pela Scene `Door.tscn`, com Signal `interacted` conectado a uma reação visível
- `InteractionComponent` do Player, funcionando sem conhecer o tipo concreto do objeto
- Um segundo objeto interativo próprio do grupo, com o mesmo contrato e um Signal próprio

Fechamento — Encontro 2

- Signal `interacted` declarado, emitido e conectado a uma reação visível
- Objeto interativo próprio do grupo funcionando com o `InteractionComponent`, sem nenhuma alteração nele
- Feedback formal sobre as soluções de interação apresentadas
- Módulo 2 avança com o par contrato + Signal sustentando todo objeto interativo futuro

Resultado Esperado da Semana

- Contrato `Interactable` implementado por ao menos uma Scene do Vertical Slice
- Signal de interação conectado a uma reação concreta
- Cada grupo com seu próprio objeto interativo (porta, alavanca ou equivalente)
- Turma relaciona contrato e Signal ao equivalente da Unity (`IInteractable` e `UnityEvent` / `event` / `Action`)

Checklist da Semana

- [] Contrato `Interactable` (`has_method("interact")`) definido e implementado em `Door.tscn`
- [] `InteractionComponent` no Player, detectando via `Area3D` e chamando `interact()`
- [] Ação `interagir` adicionada ao Input Map
- [] Signal `interacted` declarado, emitido em `interact()` e conectado a uma reação separada
- [] Segundo objeto interativo do grupo, com contrato e Signal próprios
- [] Feedback formal recebido sobre a solução de interação do grupo

Próximos Passos — Semana 6

O par contrato `Interactable` + Signal construído nesta semana é pré-requisito direto da Semana 6, que introduz `ItemData` (Resource customizado) e Enums para separar dados de design da lógica de gameplay.

Leitura recomendada: Godot Docs — Signals, GDScript (`has_method`); Unity Manual (consulta comparativa) — Events, Interfaces em C#.